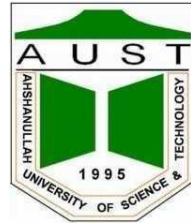


Ahsanullah University of Science & Technology
Department of Computer Science & Engineering



Project Title: A Book

Computer Graphics Lab (CSE 4204)

Final Project Report

Student ID	
Shuhrid Abrar	20220104028
Mahadir Rahaman	20220104046
Submission Date	30 September 2026

FINAL PROJECT REPORT

Project 12 — A Book

This report presents an interactive Three.js scene built around a textured Death Note and a wooden reading table. The book sits inside a reconstruction of Kira's bedroom, which gives the required lighting, projection, textures, animation, and navigation a consistent setting.

1. Problem Analysis

1.1 Project Requirements

The assigned project is “A Book.” Its minimum object requirements are a textured book and a textured table. The keyboard must move the camera around the book and change the book-cover texture; the mouse must open or close the book and flip its pages; and a light source must rotate around the book. The general course requirements additionally demand a custom shader, lighting, perspective projection, a texture for each visible object category, animation, and both mouse and keyboard interaction.

The implementation satisfies these requirements through the following design decisions:

- A modeled Death Note with a textured leather cover, paper block, binding, five rule pages, lined notebook pages, and a fountain pen.
- A procedurally textured wooden round table with four legs and two upholstered floor cushions.
- A responsive perspective camera with four authored viewpoints and bounded free movement inside a 24×18 unit room.
- An animated point light orbiting the notebook, ceiling area lights, a desk spotlight, and room fill lighting.
- A custom GLSL alcove shader, procedural canvas textures, smooth cover motion, curled two-sided page turns, and idle character animation.
- Raycast-based mouse selection and continuous keyboard controls for views, camera motion, book state, cover finish, reset, and help.

1.2 Core Integration

The models are assembled from Three.js primitives: boxes, planes, cylinders, spheres, tori, cones, lathed profiles, tubes, and rounded boxes. They are organized into separate groups for the north, west, south, and east walls. The Death Note is a separate articulated group positioned on the reading table. The front cover and turning sheet each use a hinge group, allowing their local rotations to animate independently of the notebook's world position.

The renderer uses ACES filmic tone mapping, sRGB output, soft shadows, exponential fog, and a device-aware pixel ratio. A `THREE.PerspectiveCamera`` is created with a 49° field of view, a 0.05 near plane, and an 80-unit far plane. Camera positions and targets are interpolated toward the selected Room, Desk, Death Note, or Shinigami viewpoint, while room bounds prevent the viewer from escaping the interior shell.

Lighting combines a HemisphereLight for broad ambient balance, a shadow-casting DirectionalLight, two RectAreaLights in the ceiling panels, a large perimeter RectAreaLight, a focused desk SpotLight, and the

required animated PointLight around the book. Standard and physical materials react to these lights, while emissive-looking ceiling panels and screen art use tone-mapping-independent basic materials.

Textures are generated at runtime with `CanvasTexture` for wood grain, leather grain, paper fibers, fabric weave, book spines, rule pages, notebook lines, and written names. Supplied artwork is loaded only where exact visual identity matters: the Death Note cover and the L graphic on the monitor and CRT television.

1.3 Technical Challenges

Camera motion uses exponential smoothing rather than a fixed interpolation factor. For each frame, the interpolation factor is $\alpha = 1 - e^{-k\Delta t}$, so movement remains visually consistent at different frame rates. Mouse and keyboard orbiting convert the camera offset to spherical coordinates; azimuth θ changes horizontally and polar angle ϕ changes vertically. The polar angle and final Cartesian position are clamped to keep the camera inside the room and avoid inverted views.

The orbiting light follows $x = r \cos(\omega t)$, $z = z_0 + r \sin(\omega t)$, and $y = y_0 + a \sin(0.6t)$. This produces continuous circular motion around the notebook with a small vertical oscillation. The page-turn hinge uses cubic easing so the page starts and stops naturally: $\text{rotation} = \pi \times \text{ease}(s)$. During the turn, each vertex receives an additional curl proportional to $\sin(\pi s) \sin(0.85\pi u)$, where s is turn progress and u is distance from the binding. This creates a curved sheet rather than a rigid rotating plane.

For object selection, pointer coordinates are converted from canvas pixels to normalized device coordinates in the range $[-1, 1]$. A Three.js Raycaster projects through the perspective camera and intersects the notebook, exposed page, cover, and pen meshes. Clicks are separated from drag gestures using a movement threshold, preventing an orbit operation from accidentally turning a page.

2. User Interaction

2.1 Key Interaction

Keyboard state is stored in a Set so continuous movement remains smooth while keys are held. In room views, W/S move along the horizontal camera-forward vector, A/D move along the perpendicular right vector, and Q/E lower or raise the camera. In the Death Note view, WASD and the arrow keys change spherical camera angles around the book instead of translating the entire viewpoint.

Key	Action
1 / 2 / 3 / 4	Select Room, Desk, Death Note, or Shinigami view.
W / A / S / D	Move through the room; orbit around the book in Death Note view.
Arrow keys	Orbit horizontally and vertically around the Death Note.
Q / E	Lower or raise the camera; E opens or closes the book in book view.
C or T	Cycle through black, green-black, and umber-black leather finishes.
R	Reset the active authored viewpoint.
H	Open or close the help panel.

2.2 Mouse Interaction

Dragging on the canvas orbits the camera around its current target, including full vertical inspection within safe angle limits. The mouse wheel scales the camera-to-target offset to zoom. The interface buttons switch between the four authored views. In Death Note view, clicking the cover opens or closes it, clicking an exposed page starts the page-turn animation, and clicking the fountain pen arms writing mode. A later click on a lined notebook page writes the configured names. A visible prompt always reports whether the next action will open or close the notebook.

3. Design and Aesthetic Aspects

3.1 Visual Elements

The room uses a restrained green, blue, cream, dark wood, and black palette derived from the supplied reference images. Green built-in shelves unify the north, west, and east walls; warm wooden flooring and table surfaces prevent the scene from becoming visually flat; and the blue bedding and curtains provide cool contrast. The desk, television shelf, bed alcove, cupboards, windows, closet, and air conditioner are arranged as a continuous interior rather than isolated objects.

Most surfaces use `MeshStandardMaterial` or `MeshPhysicalMaterial`, allowing roughness, metalness, clearcoat, transparency, and shadows to respond to the scene lights. Wood receives directional grain and plank seams; fabric receives a fine woven pattern; leather receives multi-scale mottling; paper receives subtle fibers and aging; and the monitor artwork remains crisp through sRGB sampling and anisotropic filtering. The custom alcove shader mixes a dark green base with a height-dependent central glow, animated pulse, and procedural grain.

The ceiling uses two luminous square panels and a recessed perimeter glow to illuminate the full room. A focused desk light provides local contrast, while the orbiting green-white book light draws attention to the main interactive object. Soft shadows, filmic tone mapping, and subtle fog help integrate the large number of modeled elements.

3.2 Snapshots



Figure 1: Complete room view showing the enclosed architecture, bed wall, reading table, ceiling panels, and Shinigami figures.



Figure 2: West-wall workstation with textured shelving, monitor, keyboard, mouse, lamp, dual drawers, and chair.



Figure 3: Open Death Note on the textured wooden reading table with the fountain pen and animated-book interface.



Figure 4: Two-sided page-turn animation; the flexible sheet arches as it rotates around the binding.

4. Code Implementation

4.1 Software Platform

The project uses HTML5 and CSS3 for the responsive interface, modern JavaScript modules for application logic, Three.js 0.160 for scene construction and rendering, WebGL for GPU output, GLSL for the custom shader, Vite for development and production bundling, and Playwright Core with Microsoft Edge for automated smoke testing. Git is used for version control, and the production build is deployable as a static Vercel site.

4.2 Custom Shaders

The vertex shader forwards each vertex's UV coordinates and local position before applying the standard Three.js model-view and projection matrices. The fragment shader calculates a vertical `topLight` gradient with `smoothstep`, a centered horizontal falloff, a time-driven pulse, and a small pseudo-random grain term. It then mixes a dark green base color with the brighter glow uniform. The `uTime` uniform is updated every frame from the main animation clock, so the alcove illumination gently breathes.

4.3 Project Feature Table

#	Feature	Status
1	Textured 3D Death Note	Implemented
2	Textured 3D wooden table	Implemented
3	Custom GLSL shader	Implemented
4	Multi-source lighting	Implemented
5	Responsive perspective projection	Implemented
6	Textures for every major object category	Implemented
7	Animated cover and curved page turns	Implemented
8	Keyboard camera orbit and movement	Implemented
9	Keyboard cover-texture change	Implemented
10	Mouse open/close and page flip	Implemented
11	Light rotating around the book	Implemented
12	Fountain-pen writing interaction	Implemented
13	Complete room and four authored viewpoints	Implemented
14	Automated render/interaction smoke test	Implemented

5. Contribution

Both group members worked on the scene plan, compared the result with the supplied references, implemented the Three.js features, tested the interactions, fixed rendering problems, and prepared the final documentation.

Student	ID	Contribution
Shuhrid Abrar	20220104028	Joint design, modeling, interaction development, testing, debugging, and report preparation.
Mahadir Rahaman	20220104046	Joint design, modeling, interaction development, testing, debugging, and report preparation.

6. Testing and Deployment

Validation includes a production Vite build and an automated Playwright smoke test. The test loads the WebGL scene, confirms that the interface becomes visible, captures the room from multiple camera angles, selects the Desk and Death Note views, opens the notebook, turns pages, exercises pen interaction, closes the book through the interface, and fails when browser console or page errors occur. The optimized static build has also been verified through a Vercel deployment with an HTTP 200 response and successful browser rendering.